
(Group 18) Dirty Swapping: Training-Free KV Cache Replacement for Efficient LLM Reasoning

Changho Choi, Yucheon Park, Hyunbeen Kim, Gyeom Hwangbo
Carnegie Mellon University (IITP Program, South Korea)
{changhoc, yucheonp, hyunbeek, ghwangbo}@andrew.cmu.edu

Abstract

We propose *Dirty Swapping*, a training-free method for revisiting past reasoning decisions without discarding subsequent computation. Short alternative rollouts are cached in advance, and a selected past key–value (KV) segment is later replaced while downstream text and states remain unchanged. We investigate whether this deliberately inconsistent state improves reasoning accuracy at an acceptable inference-time cost.

1 Overview

Motivation and Objectives. Revisiting an earlier choice by discarding completed reasoning is computationally wasteful. Our objective is to reuse stored alternatives while preserving the existing suffix, and to evaluate the accuracy–latency trade-off including the cost of preparing alternatives.

2 Related Work

Decoding-time intervention frameworks such as CARE [2] perform rollbacks upon detecting harmful content, but recomputing the discarded suffix incurs substantial overhead. Recent work shows that KV caches can instead be directly manipulated: History Swapping [1] and Memory Inception [3] demonstrate that modifying cached states can steer subsequent generation, while Contiguity, Not Importance [4] highlights the stale-cache problem caused by reusing downstream states after an edit. Building on these findings, we propose “Dirty Swapping,” which replaces selected past states with pre-cached Top- k counterfactual alternatives while leaving downstream KV states unchanged, thereby avoiding rollback and suffix recomputation.

3 Methodology

Model Description. We will use Qwen3-4B-Thinking-2507 with frozen weights. We will replace token-indexed KV entries across the attention layers. At selected positions, we will generate short rollouts from the top- k candidate tokens under the same prefix and store their KV segments. The highest-probability candidate defines the main continuation. After further reasoning, we will overwrite only the chosen past segment with a stored alternative. Given $ABCD$, for example, the KV entries for BC are replaced by those for EF , previously computed under A . The text and KV entries for D remain unchanged: no suffix is recomputed or regenerated. This inconsistent cache differs from a freshly computed cache for $AEFD$. Initially, we will apply one replacement per trace using equal-length segments at identical positions.

Dataset. We will evaluate on AIME25 and HMMT25 for competition-level mathematics, GSM8K for multi-step arithmetic, and GPQA Main for scientific reasoning. Development examples will be disjoint from final evaluation examples.

Evaluation Metric. We will report final-answer accuracy and per-example inference time from input processing to final-answer generation, including alternative rollouts and KV replacement. Comparisons will match problems, main-path generation rules, decoding limits, and seeds.

Loss Function. The method is training-free, with no loss function, parameter updates, or additional training cost. Additional computation occurs only at test time.

Experimental Depth and Iteration. Initial settings are $k = 4$, 32-token rollouts, and a 128-token replacement delay. We will first use fixed positions and alternatives, then vary the configs.

4 Baseline and Extensions

Baseline Selection and Evaluation. The autoregressive baseline will follow the same main-path generation rules without alternative rollouts or KV replacement. We will compare accuracy and inference time, including all preparatory rollouts in the proposed method’s cost.

Implemented Extensions/Experiments. We will implement intermediate replacement using stored counterfactual KV segments and compare it with the baseline, preserving all existing downstream.

Baseline Reproduction Evidence. Reproduction under these settings is pending. We will retain per-example outputs, execution logs, accuracy, and inference times.

5 Results and Analysis

Results. We will evaluate whether sparse replacement is useful despite preserving stale downstream states, and characterize the resulting accuracy–latency trade-off.

Failure Case Analysis. We will examine contradictions or repetition caused by downstream KV entries retaining the original path, and failures caused by poor alternatives.

Ablation Analysis. We will vary rollout length, candidate count, and replacement delay, comparing settings with each other and with the baseline.

6 Potential Challenges and Risks

Replacing a past KV segment may introduce inconsistencies because downstream KV entries will still reflect the original context. These retained states could limit or override the influence of the replacement, resulting in little or no change in subsequent generation. Alternatively, the mismatch could disrupt contextual coherence and lead to inconsistent or incoherent outputs.

7 Future Directions

Implementing KV Cache Replacement. We will implement a cache-editing module that stores alternative KV segments and overwrites only the selected attention-cache entries, verifying that downstream KV entries remain unchanged.

Where to Prepare Alternative Rollouts. Exploring every position is prohibitively expensive. We will investigate fixed intervals and uncertainty-based criteria for selecting useful rollout positions under a limited inference budget.

How to Evaluate Alternatives. We will develop a metric, such as a verifier score, to judge whether a stored alternative is better than the original choice, and test whether it predicts improved accuracy. Ground-truth answers will be used only for evaluation, not intervention selection.

Advanced: Inference-Time Optimization. We will investigate inference runtimes such as vLLM for batched rollouts, shared-prefix reuse, and efficient KV-block updates, adding cache-editing hooks where needed for selective KV replacement.

References

- [1] Mukkesh Ganesh, Kaushik Iyer, and Arun Baalaji Sankar Ananthan. Whose narrative is it anyway? a kv cache manipulation attack. *arXiv preprint arXiv:2511.12752*, 2025.
- [2] Xiaomeng Hu, Fei Huang, Chenhan Yuan, Junyang Lin, and Tsung-Yi Ho. Care: Decoding-time safety alignment via rollback and introspection intervention. In *Advances in Neural Information Processing Systems*, volume 38, pages 56932–56967, 2025.
- [3] Andy Zeyi Liu, Michael Zhang, Ilana Greenberg, Adam Alnasser, Lucas Baker, and John Sous. Memory inception: Latent-space kv cache manipulation for steering llms. *arXiv preprint arXiv:2605.06225*, 2026.
- [4] Mingyang Mao, Wyatt Mackey, and Xiaomin Lin. Contiguity, not importance: Budgeted repair of stale kv caches after document edits. *arXiv preprint arXiv:2609.17983*, 2026.